[Dinu Barbu]

What happens when an agent reads your prompt.

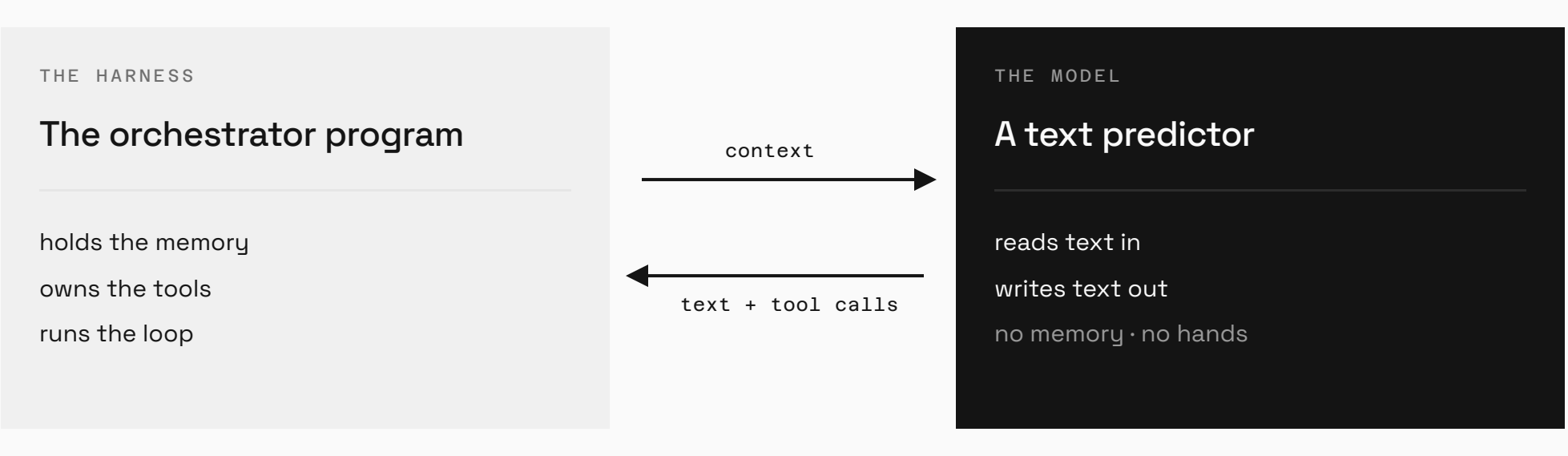
Six sections, one idea: an agent is a program passing text to a text predictor, over and over. Everything that feels mysterious about it — memory, tools, context limits, cost — falls out of that one arrangement.

// an explainer · ↳ dvinubius

THE MENTAL MODEL · KEEP THIS ONE

What is an agent

An agent is a conversation between a **harness** (the orchestrator) and a **model** (a text predictor). The harness hands over a big blob of text; the model hands back text. That exchange is the whole game.



Ink is the model. Shade is the harness. That holds for every figure below.

Everything else in this explainer is just detail on those two arrows: what's **in** the context the harness sends, and what the harness **does** with the text it gets back.

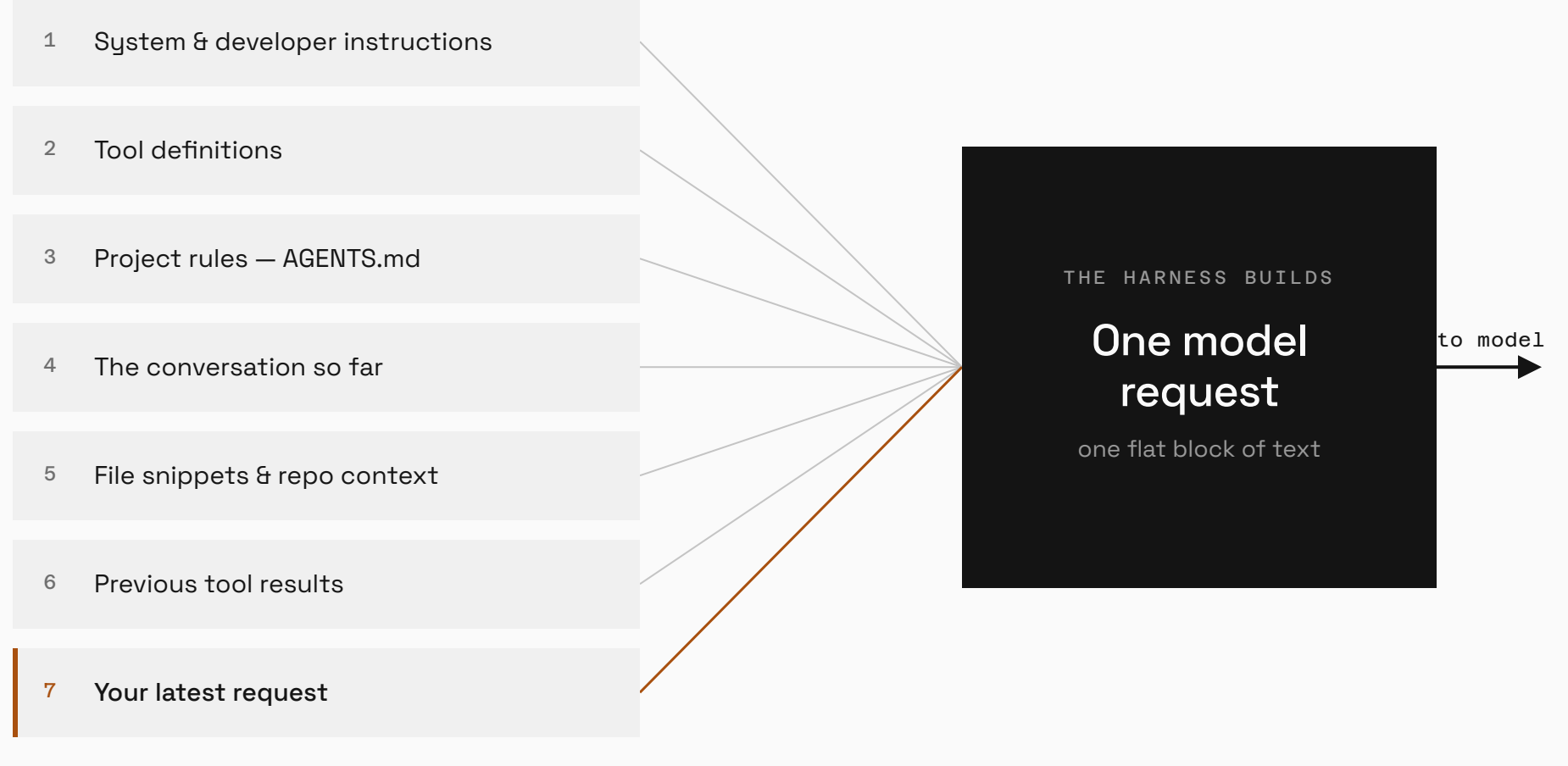
THE FIRST ARROW · WHAT THE MODEL IS HANDED

"Context" is everything the model sees

The model has no window into your machine. Before every call, the harness assembles one request — a single stack of text. If something isn't in this stack, the model simply doesn't know it exists.

BUILDING THE REQUEST

The harness pulls from several places and flattens them into one block of text. Think of these as **labels for what goes in** — not seven tidy, separate boxes.



Here's what each label points at:

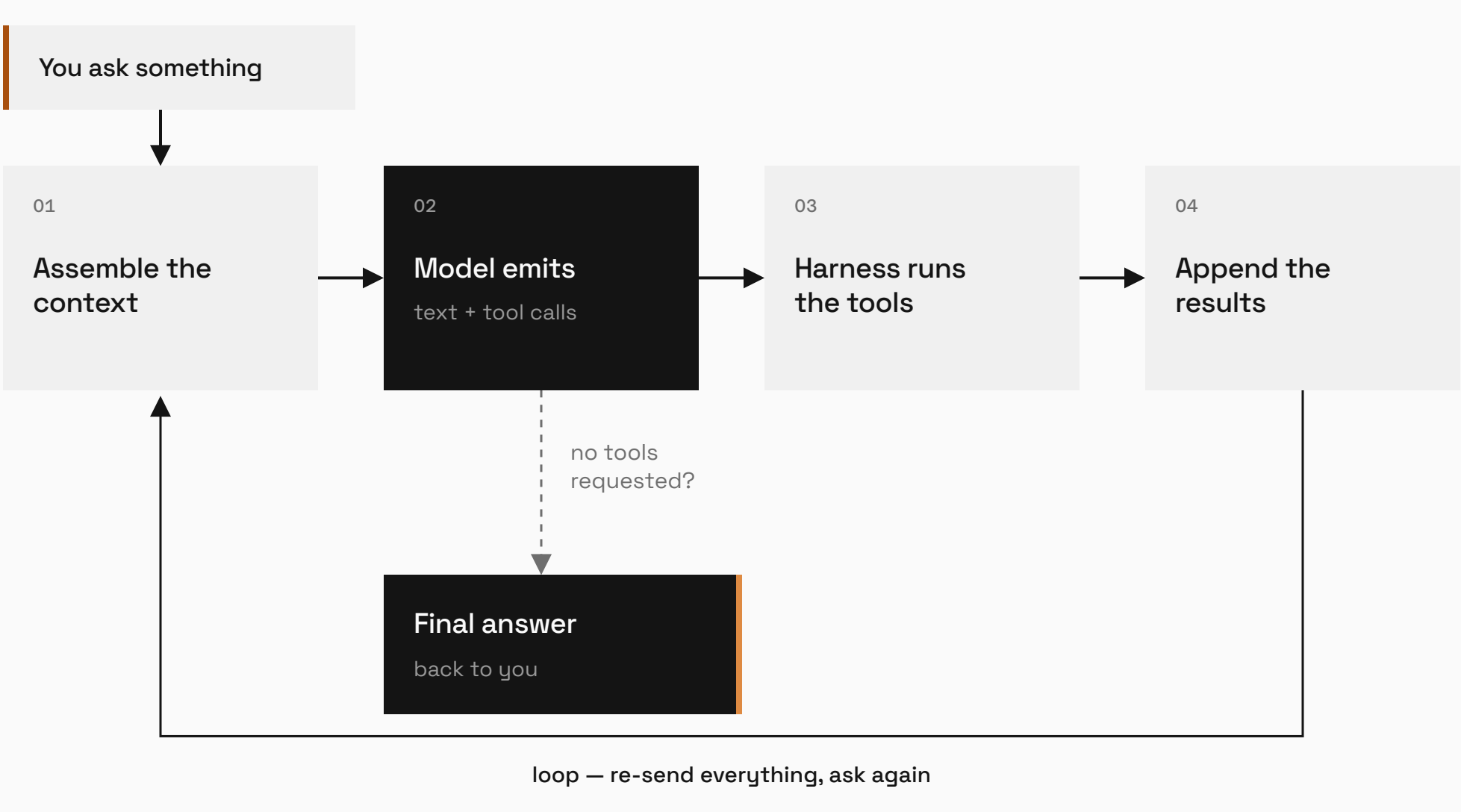
one model request	ASSEMBLED FRESH EACH TURN
1	**System & developer instructions** — the model's standing rules — how to behave, when to stop
2	**Tool definitions** — the menu of actions it can request — read, search, run, edit
3	**Project rules** `AGENTS.md, etc.` — your repo's conventions, do's and don'ts
↓ ITEMS 4–7 ARE REALLY ONE THING: THE CONVERSATION	
4	**The conversation so far** — the whole back-and-forth this session — and it already *contains* 5 and 6
5	**File snippets & repo context** `part of #4` — code the model pulled — it arrived earlier as a tool result
6	**Previous tool results** `part of #4` — outputs, errors, diffs from earlier tool runs
7	**Your latest message** `newest line of #4` — the new thing you just asked for

> **These labels overlap — they're a simplification, not seven disjoint boxes.**
> Only 1–3 are truly separate. Everything else is one conversation: #4 is the whole back-and-forth, which already holds the file snippets (#5) and tool results (#6) the model pulled, with your latest message (#7) as its newest line. We split them out only to name what tends to go in. The next section shows the real, non-overlapping layout — stable prefix on top, one ordered conversation below.

THE SIGNATURE MOVE · THE LOOP

One prompt becomes many turns

The reply usually isn't an answer yet — it's a request to **do** something. So the harness acts, folds the result back into the context, and asks again. Around and around until the model has nothing left to ask for.



Steps 1 & 4 are bookkeeping
The harness builds the text going in and stitches results back in after.

Step 2 is the only model call
It thinks once, then yields control. It can't act on its own.

Step 3 is the real work
Files get read, tests get run, edits get applied — all by the harness.

THE DETAIL THAT MAKES IT CLICK

"Memory" is the harness re-feeding the whole transcript

Because the model forgets everything between calls, the harness can't just send the new result. It re-sends the **entire growing history** every single turn. Each call is bigger than the last.

Ember marks the only genuinely new text in each request — the tool result the harness just handed back.

> This is why agents have a **context limit** and why long sessions slow down or start "forgetting" early details — the transcript eventually gets too big to re-send in full, so the harness has to trim or summarize it.

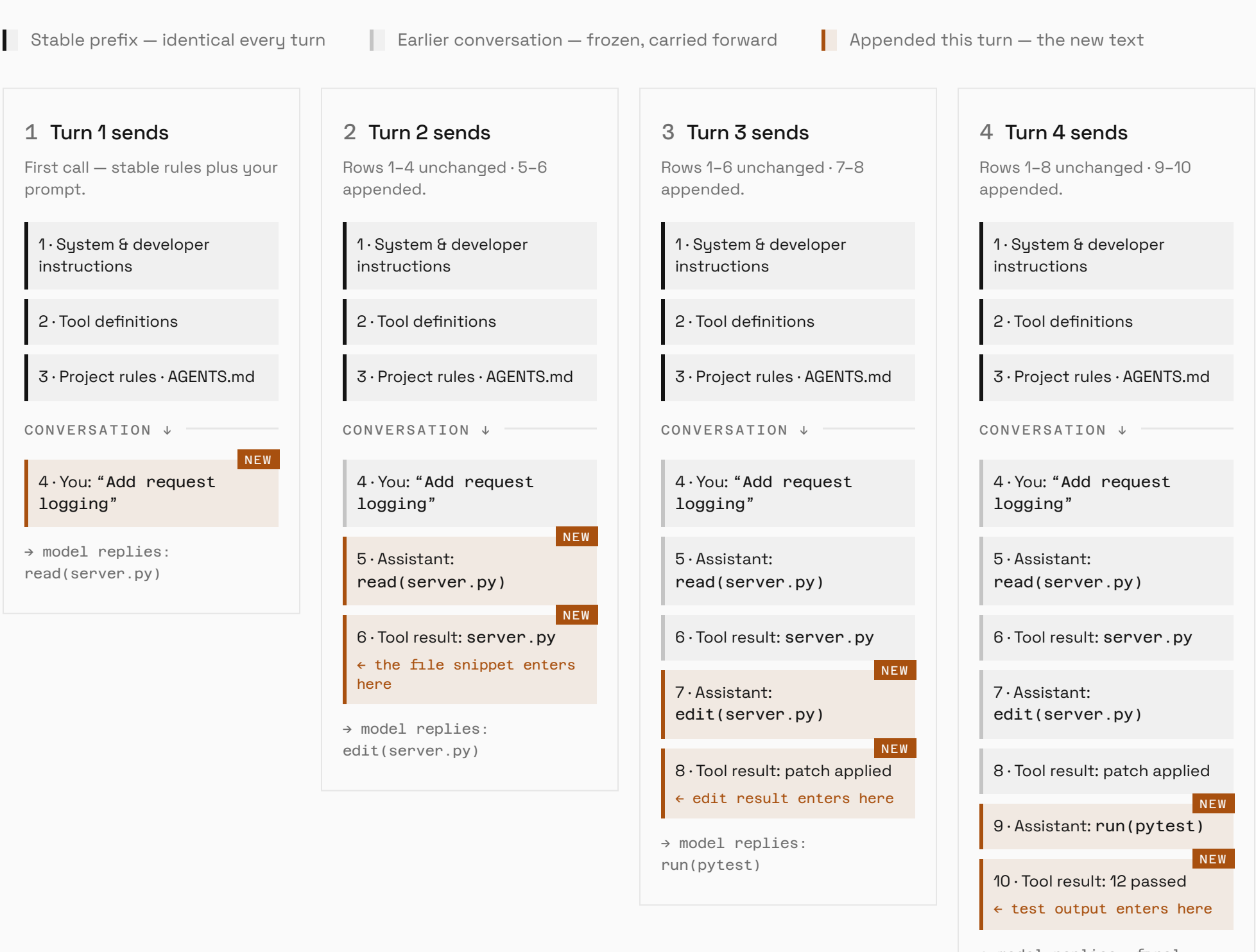
LOOK CLOSER · ONE REQUEST, FULLY EXPANDED

What each turn actually sends

The seven sources from before aren't seven floating slots. The harness lays them out as **one ordered sequence**: the stable three at the top, then the conversation — and the conversation absorbs file snippets and tool results as it grows.

One full cycle of the example `Add request logging to the server` — four model calls, then a final answer. Watch the request grow by appending only; earlier rows never change.

Stable prefix — identical every turn | Earlier conversation — frozen, carried forward | Appended this turn — the new text



↳ Turn 4's reply closes the cycle

```
11 · Assistant (final): "Added logging middleware — 12 tests pass."
no tool call → the harness stops looping and hands this back to you
```

Row 11 is the model's reply to Turn 4 — it isn't **sent** in another request; it's the output that ends the loop. Whatever you type next appends as **row 12**, and the harness fires Turn 5 on top of all eleven rows — the same cycle, continued.

> **So "the conversation so far" is everything from row 4 down — and that's the part that confuses people: it isn't only your messages.**
> The model's own replies, the files it read, and every tool result are all entries in that one growing list. "File snippets & repo context" and "previous tool results" aren't separate slots beside the conversation — they **enter** the conversation as tool-result messages, in order, and then never change.

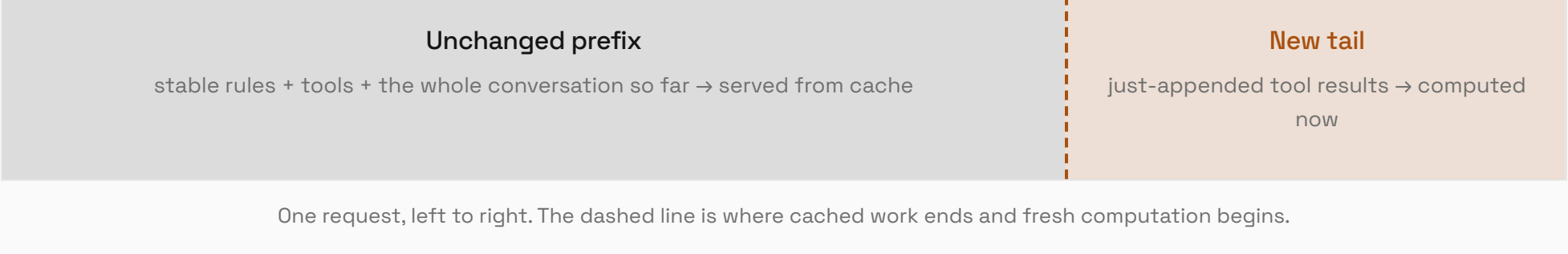
One simplification: this example does one tool call per turn (read, then edit, then test) to keep the steps clear. A real agent often batches several tool calls into a single reply, which collapses this into fewer turns. The append-only growth pattern is identical either way — only the number of turns changes.

THE PAYOFF · WHY APPEND-ONLY MATTERS

Why the prefix has to stay byte-for-byte identical

A model normally re-reads its entire input from scratch on every call. The **KV cache** is how it avoids that: for every token it has already processed, it stores the intermediate attention values — the **keys** and **values**. Hand it the exact same opening sequence again and it skips straight past those tokens, computing only what's new at the end.

That's the real reason the harness re-sends everything unchanged and only appends. Each turn, the long stable prefix **and** the entire prior conversation are a cache hit. Because the model already computed its own replies while generating them, the only genuinely new text it has to read each turn is the **tool results** the harness just handed back.



One request, left to right. The dashed line is where cached work ends and fresh computation begins.

> **Change anything *early* — edit AGENTS.md or the system prompt, reorder messages, or trim and summarize old turns to fit the context limit — and every cached value after the change is invalid.**
> The model has to recompute from that point on. **Append-only is cheap; rewriting history is expensive.** It's also why editing AGENTS.md mid-session is costly: it sits near the top, so the whole conversation below it falls out of cache — and why trimming a long conversation (the previous section) costs more than it looks.